BCS755A | Modules 3–5 Question & Answer Bank

FOCUSED VTU EXAMINATION RESOURCE

59 fully answered questions | 5-mark and 10-mark formats

Relational Model • SQL and Normalization • Advanced SQL and Concurrency

Model answers, worked relational algebra and SQL, marking guidance and practice papers

Seventh Semester • Updated November 2026

BCS755A | INTRODUCTION TO DBMS

VTU-Aligned Question Bank and Model Answers | Modules 3, 4 & 5

Seventh Semester | BCS755A | Modules 3–5 only | November 2026

> Examination note. These are detailed, independently prepared model answers with suggested point-wise marking elements; they are not an official VTU valuation key. Students should write all requested steps, exact relational algebra and SQL expressions, worked examples and diagrams where relevant. The bank covers Modules 3, 4 and 5 exclusively.

How to use the model answers

Item	Details
Course	BCS755A — Introduction to DBMS
Included content	Modules 3, 4 and 5 only
Answered question count	59 — Module 3: 17, Module 4: 20, Module 5: 22
Answer format	5-mark and 10-mark solutions, SQL and relational algebra
Practice	Two focused 60-mark university-style papers and two 25-mark tests

For 5-mark theory questions, aim for a precise definition, three or four distinct technical points, and a relevant example or diagram. For 10-mark questions, add labeled algorithms/diagrams, sufficient reasoning and a worked example. Suggested marking allocations are learning aids; the actual examiner may accept equivalent correct solutions. For five-part query questions, attempt all five expressions or queries rather than developing only one or two.

Module coverage at a glance

Module	Coverage
3	Relational model concepts, constraints, update operations, relational algebra, division and grouping
4	SQL DDL and DML; functional dependencies; normalization from 1NF through 5NF
5	Advanced SQL, views, assertions and triggers, concurrency control, locking, timestamps, MVCC and validation

MODULE 3: Relational Model and Relational Algebra

17 fully answered questions in this module.

Question index

ID	Question	Marks
M3-Q01	Define relation, domain, tuple, attribute, degree, cardinality and relation schema. Explain relational characteristics.	10
M3-Q02	Explain relational model constraints with examples.	10
M3-Q03	Explain how INSERT, DELETE and UPDATE operations deal with constraint violations, with examples.	10
M3-Q04	Explain SELECT, PROJECT and RENAME relational algebra operations.	5
M3-Q05	Explain UNION, INTERSECTION, SET DIFFERENCE and CARTESIAN PRODUCT.	5
M3-Q06	Explain theta join, equijoin, natural join, outer join and self join.	10
M3-Q07	Define the DIVISION relational algebra operation and explain it with a complete example.	10
M3-Q08	Explain aggregate/grouping operations and generalized projection in relational algebra.	5
M3-Q09	Write five COMPANY relational algebra expressions: Research employees, Smith-related projects, managers, salary statistics and project staffing counts.	10
M3-Q10	Write relational algebra for Bob’s department, departments with projects, Alpha projects, departments without projects and IT employees.	5
M3-Q11	Write relational algebra for HR employees, salary conditions, IT-project departments, mean salary and employee-department join.	5
M3-Q12	Explain relational algebra query trees and why expression rewriting matters.	10
M3-Q13	Differentiate superkey, candidate key, primary key, alternate key, composite key and foreign key.	5
M3-Q14	Explain transaction concept and relation-state consistency.	5
M3-Q15	Illustrate relational algebra for employees without dependents and departments with no employees.	5
M3-Q16	Write relational algebra to find employees earning more than their supervisors.	5
M3-Q17	Explain union compatibility and list a complete set of basic relational algebra operations.	5

Model answers and suggested marking elements

M3-Q01 | 10 marks

Question. Define relation, domain, tuple, attribute, degree, cardinality and relation schema. Explain relational characteristics.

Suggested marking allocation: Definitions 5; formal notation 2; characteristics 3.

The **relational model** represents data using relations (tables). A **domain** is a set of atomic permitted values; an **attribute** is a named column; a **tuple** is a row; a **relation schema** R(A1,…,An) is a relation name and its attribute list. A **relation state** r(R) is a set of valid tuples at a particular time. **Degree/arity** is number of attributes; **cardinality** is number of tuples. Formally: $r(R) \subseteq dom(A1) \times \ldots \times dom(An)$.

Characteristics: (1) duplicate tuples are not allowed in the mathematical relation; (2) tuple ordering is immaterial; (3) attribute ordering follows a named schema, while queries can use attribute names; (4) values are atomic in first-normal-form relations; (5) every attribute has a domain; (6) NULL may represent absent/unknown/inapplicable data; (7) candidate and primary keys uniquely identify tuples. **Example:** STUDENT(USN, Name, CGPA) has degree 3; a current state with 50 student rows has cardinality 50. SQL tables may permit duplicate rows without DISTINCT, unlike the mathematical set definition.

M3-Q02 | 10 marks

Question. Explain relational model constraints with examples.

Suggested marking allocation: Domain 2; key 2; entity integrity 2; referential integrity 2; example/other constraints 2.

1. **Domain constraints (2):** each attribute value must belong to its declared domain. Example: CGPA DECIMAL(3,2) CHECK(CGPA BETWEEN 0 AND 10); Age must be integer in an allowed range.
2. **Key constraints (2):** a candidate key uniquely identifies tuples; no two rows may have the same candidate-key values. One candidate key is chosen as **primary key**; others become alternate keys. EMPLOYEE(Ssn PRIMARY KEY).
3. **Entity integrity (2):** no component of a primary key may be NULL. Example: an EMPLOYEE row with Ssn=NULL is rejected.
4. **Referential integrity (2):** a non-NULL FK must match a referenced candidate/primary-key value. Example: EMPLOYEE.Dno REFERENCES DEPARTMENT(Dnumber); Dno=99 is invalid if department 99 is absent.
5. **Semantic/business constraints (2):** rules such as Salary >= 0, employee start date <= today, or max credit load. Use CHECK, triggers or assertions where supported.

Diagram/instance: DEPARTMENT(10,‘IT’); EMPLOYEE(‘E1’,‘Anu’,10) is valid; EMPLOYEE(‘E2’,‘Bob’,99) violates FK. Distinguish key uniqueness from entity integrity’s NOT NULL rule.

M3-Q03 | 10 marks

Question. Explain how INSERT, DELETE and UPDATE operations deal with constraint violations, with examples.

Suggested marking allocation: INSERT violations 3; DELETE 2; UPDATE 2; remedies/examples 3.

INSERT: may violate **domain** (Salary='abc'), **key** (duplicate Ssn), **entity integrity** (Ssn=NULL), or **referential integrity** (Dno points to nonexistent department). Remedy: reject transaction, correct values, or insert referenced parent first.

DELETE: deleting DEPARTMENT(10) when EMPLOYEE rows reference it may break referential integrity. Options configured on FK: RESTRICT/NO ACTION (reject), CASCADE (delete children), or SET NULL/SET DEFAULT (if permitted and valid). SET NULL is not valid for a child column constrained NOT NULL.

UPDATE: setting EMPLOYEE.Ssn to a duplicate violates key constraints; updating Dno to unknown value violates FK; changing a referenced DEPARTMENT.Dnumber may require ON UPDATE CASCADE/RESTRICT/SET NULL. Updating Salary outside domain violates CHECK.

Example: DELETE FROM DEPARTMENT WHERE Dnumber=10; is rejected under RESTRICT if active employees reference 10. **Transaction safety:** DBMS must leave constraints satisfied at commit; rejected statements are rolled back according to transaction/DBMS behavior. Specify which relation is parent/child for each example.

M3-Q04 | 5 marks

Question. Explain SELECT, PROJECT and RENAME relational algebra operations.

Suggested marking allocation: Selection 2; projection 2; rename/example 1.

Selection σ_condition(R) filters rows, preserving all columns: σ_Salary>60000(EMPLOYEE). **Projection** π_A,B(R) chooses columns and eliminates duplicate resulting tuples under set semantics: π_Name,Dno(EMPLOYEE). **Rename** ρ_NewName(NewA,NewB)(R) changes the relation/attribute labels; useful for self-joins. **Combination:** π_Name(σ_Dno=10(EMPLOYEE)) retrieves names in department 10. Selection is a horizontal subset, projection a vertical subset; both are unary operations.

M3-Q05 | 5 marks

Question. Explain UNION, INTERSECTION, SET DIFFERENCE and CARTESIAN PRODUCT.

Suggested marking allocation: Three set operations + compatibility 3; product 2.

For union-compatible relations R and S (same number of attributes, corresponding domains compatible): **union** $R \cup S$ includes tuples in either; **intersection** $R \cap S$ includes tuples in both; **difference** $R - S$ includes tuples in R but not S. All use set semantics (duplicates removed). **Cartesian product** $R \times S$ pairs every tuple of R with every tuple of S, yielding $|R| \times |S|$ tuples and degree degree(R)+degree(S). Example: 3 employees × 2 projects produces 6 pairs. Cross product needs no union compatibility.

M3-Q06 | 10 marks

Question. Explain theta join, equijoin, natural join, outer join and self join.

Suggested marking allocation: Join definitions 5; examples 3; outer/self comparison 2.

Theta join: $R \bowtie_\theta S = \sigma_\theta(R \times S)$; theta condition uses $=, <, >, \leq, \geq, \neq$. Example: EMPLOYEE ⋈_{Dno=Dnumber} DEPARTMENT.

Equijoin: special theta join in which all comparisons are equality; duplicate join columns may remain.

Natural join: equijoin on equally named attributes, with duplicate common attributes eliminated. Rename columns first if same names have different meanings.

Left outer join (⟕): retains all left rows; unmatched right attributes are NULL. **Right outer (⟖)** and **full outer (⟗)** similarly preserve their sides/all sides.

Self join: two renamed copies of the same relation, such as E1=ρ_E1(EMPLOYEE), E2=ρ_E2(EMPLOYEE); σ_E1.Super_ssn=E2.Ssn(E1×E2) identifies employee-supervisor pairs.

Example: a department without employees disappears from an inner join but remains with NULL employee fields in DEPARTMENT ⟕_{Dnumber=Dno} EMPLOYEE.

M3-Q07 | 10 marks

Question. Define the DIVISION relational algebra operation and explain it with a complete example.

Suggested marking allocation: Purpose/requirements 2; example tables 3; division result 2; equivalent algebra 3.

Meaning: R(X,Y) ÷ S(Y) returns X-values related to **every** Y-value in S. This models universal quantification ("all"). Attributes of S must correspond to a subset of attributes of R.

Example: WORKS_ON(Essn, Pno): (A,P1), (A,P2), (B,P1), (C,P1), (C,P2). REQUIRED(Pno): (P1), (P2). Thus WORKS_ON ÷ REQUIRED = {A,C} because both employees work on **both** required projects; B does not.

Using basic relational algebra:

Candidates = π_Essn(WORKS_ON)

Missing = π_Essn((Candidates × REQUIRED) − WORKS_ON)

Answer = Candidates − Missing

The cross product lists every required employee-project combination, subtracting WORKS_ON identifies missing assignments, and subtracting their employees leaves those satisfying **all** requirements. For the COMPANY schema, use π_Essn,Pno(WORKS_ON) ÷ π_Pnumber(σ_Dnum=5(PROJECT)) after renaming Pnumber -> Pno, if looking for employees assigned to all department-5 projects. Note: if REQUIRED is empty, every candidate qualifies in the standard set formulation.

M3-Q08 | 5 marks

Question. Explain aggregate/grouping operations and generalized projection in relational algebra.

Suggested marking allocation: Aggregation/notation 2; example 2; generalized projection 1.

Extended relational algebra uses **grouping/aggregation** γ_group-attributes; aggregate(expr)->alias(R). Example: γ_Dno; AVG(Salary)->AvgSal, COUNT(*)->EmpCount(EMPLOYEE) returns one tuple per department. Without grouping attributes, γ_SUM(Salary)->Total(EMPLOYEE) returns one overall aggregate. Common aggregates: COUNT, SUM, AVG, MIN, MAX. **Generalized projection** calculates expressions: π_Name,Salary*1.10->NewSalary(EMPLOYEE) (notation varies by text). Grouping is an extension, not part of the minimal relational algebra operations.

M3-Q09 | 10 marks

Question. Using the given COMPANY relational schema, write relational algebra expressions for five queries concerning departments, projects, managers and salary aggregates.

Suggested marking allocation: Five correct expressions, 2 each.

Assume EMPLOYEE(Fname,Minit,Lname,Ssn,Bdate,Address,Sex,Salary,Super_ssn,Dno); DEPARTMENT(Dname,Dnumber,Mgr_ssn,Mgr_start_date); PROJECT(Pname,Pnumber,Plocation,Dnum); WORKS_ON(Essn,Pno,Hours); DEPENDENT(Essn,Dependent_name,Sex,Bdate,Relationship).

(i) Name and address of Research-department employees: π_Fname,Lname,Address(σ_Dname='Research'(EMPLOYEE ⋈_{Dno=Dnumber} DEPARTMENT)).

(ii) Project numbers involving an employee named Smith as department manager OR project worker: ManagerP = π_Pnumber(σ_Lname='Smith'(EMPLOYEE ⋈_{Ssn=Mgr_ssn} DEPARTMENT ⋈_{Dnumber=Dnum} PROJECT)). WorkerP = ρ_Pnumber←Pno(π_Pno(σ_Lname='Smith'(EMPLOYEE ⋈_{Ssn=Essn} WORKS_ON))). Answer = ManagerP ∪ WorkerP.

(iii) Managers with at least one dependent: π_Fname,Lname(EMPLOYEE ⋈_{Ssn=Mgr_ssn} DEPARTMENT ⋈_{Ssn=Essn} DEPENDENT).

(iv) Sum, maximum, minimum and average salary: γ_SUM(Salary)->Total,MAX(Salary)->Highest,MIN(Salary)->Lowest,AVG(Salary)->Average(EMPLOYEE).

(v) Number/name/count of employees per project (including zero workers): γ_Pnumber,Pname; COUNT(Essn)->EmployeeCount(PROJECT ⟕_{Pnumber=Pno} WORKS_ON).

Use an outer join for (v) if every project must appear; do not use COUNT(*) because an unmatched outer row would count as one.

M3-Q10 | 5 marks

Question. Given EMPLOYEE(EID,Name,DeptID,Salary), DEPARTMENT(DeptID,DeptName) and PROJECT(PID,ProjectName,DeptID), write relational algebra for: (A) employees in Bob's department, (B) departments with projects, (C) employees in departments handling Alpha, (D) employees in departments without projects, (E) employees in the IT department.

Suggested marking allocation: Five expressions, 1 each.

Use EMPLOYEE(EID,Name,DeptID,Salary), DEPARTMENT(DeptID,DeptName), PROJECT(PID,ProjectName,DeptID). Set B = π_DeptID(σ_Name='Bob'(EMPLOYEE)).

A. Same department as Bob: EMPLOYEE ⋈_{EMPLOYEE.DeptID=B.DeptID} B (rename B's DeptID if needed; includes Bob unless question says *other* employees).

B. Departments with a project: π_DeptName(DEPARTMENT ⋈_{DEPARTMENT.DeptID=PROJECT.DeptID} PROJECT).

C. Employees in department handling Alpha: EMPLOYEE ⋈_{EMPLOYEE.DeptID=P.DeptID} ρ_P(π_DeptID(σ_ProjectName='Alpha'(PROJECT))).

D. Employees in departments with no projects: EMPLOYEE − π_EID,Name,DeptID,Salary(EMPLOYEE ⋈_{EMPLOYEE.DeptID=PROJECT.DeptID} PROJECT).

E. If “IT” denotes the department name: π_EMPLOYEE.*(EMPLOYEE ⋈_{EMPLOYEE.DeptID=DEPARTMENT.DeptID} σ_DeptName='IT'(DEPARTMENT)). Schema note: DEPARTMENT has no Location attribute, so a literal condition “located in IT” cannot be evaluated. If a Location attribute is supplied, use Location='IT' instead.

M3-Q11 | 5 marks

Question. Given EMPLOYEE(EID,Name,DeptID,Salary), DEPARTMENT(DeptID,DeptName) and PROJECT(PID,ProjectName,DeptID), write expressions for: (A) HR employee names, (B) employees earning over 6000, (C) employees in IT departments handling projects, (D) departmental average salary, (E) employee details with department name.

Suggested marking allocation: Five expressions, 1 each.

Interpret the clearly misprinted DEPARTMENT schema as DEPARTMENT(DeptID,DeptName); EMPLOYEE(EID,Name,DeptID,Salary), PROJECT(PID,ProjectName,DeptID). Note that no WORKS_ON relation is provided.

A. Names in HR: π_Name(EMPLOYEE ⋈_{EMPLOYEE.DeptID=DEPARTMENT.DeptID} σ_DeptName='HR'(DEPARTMENT)).

B. Salaries above 6000: π_Name(σ_Salary>6000(EMPLOYEE)).

C. Employees in departments handling IT projects: π_Name(EMPLOYEE ⋈_{EMPLOYEE.DeptID=DEPARTMENT.DeptID} σ_DeptName='IT'(DEPARTMENT) ⋈_{DEPARTMENT.DeptID=PROJECT.DeptID} PROJECT); this does **not** prove personal project assignment.

D. Average departmental salary: γ_DeptID; AVG(Salary)->AvgSalary(EMPLOYEE).

E. Employee details and department name: π_EID,Name,EMPLOYEE.DeptID,Salary,DeptName(EMPLOYEE ⋈_{EMPLOYEE.DeptID=DEPARTMENT.DeptID} DEPARTMENT).

M3-Q12 | 10 marks

Question. Explain relational algebra query trees and why expression rewriting matters.

Suggested marking allocation: Definition 2; example tree 3; equivalences 3; optimization 2.

A **query tree** represents relational operators as interior nodes and input relations as leaf nodes. Operations flow from leaves towards root. For: π_Fname,Lname(σ_Dname='Research'(EMPLOYEE ⋈_{Dno=Dnumber} DEPARTMENT)), the tree is:

```
  π_Fname,Lname
       |
σ_Dname='Research'
       |
⋈ (Dno=Dnumber)
```

```
   /     \
EMPLOYEE   DEPARTMENT
```

Rewrite: apply selection early on DEPARTMENT: π_Fname,Lname(EMPLOYEE ⋈_{Dno=Dnumber} σ_Dname='Research'(DEPARTMENT)). This reduces rows entering a join. **Equivalences:** consecutive selections commute; selections can be combined as σ_p(σ_q(R)) = σ_(p AND q)(R); selections involving only one join input may be pushed to that input; projection can be pushed down only if join/filter attributes are retained. **Benefit:** fewer intermediate tuples, lower memory/I/O, same result under valid equivalence laws. A query tree indicates *what* algebra operations are used, while an execution plan also selects physical algorithms/indexes.

M3-Q13 | 5 marks

Question. Differentiate superkey, candidate key, primary key, alternate key, composite key and foreign key.

Suggested marking allocation: Six accurate terms with consistent example 5.

Superkey: any attribute set that uniquely identifies rows (may contain extra attributes). **Candidate key:** minimal superkey, e.g., USN and Email if both unique and non-NULL. **Primary key:** selected candidate key, e.g., USN. **Alternate key:** candidate key not chosen, e.g., Email. **Composite key:** key with multiple attributes, e.g., ENROLLMENT(USN,CourseID). **Foreign key:** attribute(s) referring to a key in another or same table, e.g., ENROLLMENT.USN -> STUDENT.USN. A foreign key need not be unique and may be NULL unless constrained NOT NULL; a primary key cannot contain NULL.

M3-Q14 | 5 marks

Question. Explain transaction concept and relation-state consistency.

Suggested marking allocation: Definition 1; ACID 3; example 1.

A **transaction** is a logical unit of database work that takes a consistent state to another consistent state if it commits successfully. **Atomicity:** all operations commit or none. **Consistency:** declared integrity constraints are preserved. **Isolation:** concurrent execution behaves according to the selected isolation guarantee. **Durability:** committed changes survive crashes. **Example:** transfer ₹500 from A to B requires decrement of A and increment of B in one transaction; if the second update fails, rollback must restore A.

M3-Q15 | 5 marks

Question. Illustrate relational algebra for employees without dependents and departments with no employees.

Suggested marking allocation: Two set-difference expressions 4; explanation 1.

Employees with no dependent: π_Ssn,Fname,Lname(EMPLOYEE) − π_Ssn,Fname,Lname(EMPLOYEE ⋈_{Ssn=Essn} DEPENDENT). **Departments with no employee:** π_Dnumber,Dname(DEPARTMENT) − π_Dnumber,Dname(DEPARTMENT ⋈_{Dnumber=Dno} EMPLOYEE). **Reason:** subtract entities that have at least one matching row. Both sides of each difference must be union-compatible; include the same attribute list in the same order.

M3-Q16 | 5 marks

Question. Write relational algebra to find employees earning more than their supervisors.

Suggested marking allocation: Aliases 1; condition 2; projection/explanation 2.

Rename two copies of EMPLOYEE: $E = \rho_E(EMPLOYEE)$ and $S = \rho_S(EMPLOYEE)$; then $\pi_{E.Fname,E.Lname}(\sigma_{E.Super_ssn=S.Ssn \text{ AND } E.Salary>S.Salary}(E \times S))$. Alternatively use an equijoin with condition $E.Super_ssn=S.Ssn$ followed by salary selection. **Important:** aliasing is necessary to distinguish same-named attributes in a self join; employees with no supervisor have no matching S row.

M3-Q17 | 5 marks

Question. Explain union compatibility and list a complete set of basic relational algebra operations.

Suggested marking allocation: Compatibility 2; primitive operators 3.

Union compatibility requires the same number of attributes and type-compatible corresponding domains (attribute names may be aligned by renaming). $STUDENT(USN,Name)$ and $FACULTY(Fid,Name)$ can be unioned after renaming if ID domains are compatible. A **complete basic set** is SELECT σ, PROJECT π, UNION $\cup$, SET DIFFERENCE $-$, CARTESIAN PRODUCT $\times$, and RENAME ρ. INTERSECTION, JOIN and DIVISION can be expressed using these basic operations. Example $R \cap S = R - (R - S)$.

MODULE 4: SQL and Database Normalization

20 fully answered questions in this module.

Question index

ID	Question	Marks
M4-Q01	Explain SQL data-definition commands and demonstrate CREATE TABLE with integrity constraints.	10
M4-Q02	Classify SQL data types with examples.	5
M4-Q03	Explain INSERT, DELETE and UPDATE SQL statements with suitable examples.	10
M4-Q04	What is normalization? Explain 1NF, 2NF and 3NF with a complete example.	10
M4-Q05	Explain multivalued dependencies, 4NF, join dependencies and 5NF with examples.	10
M4-Q06	Define BCNF, compare it with 3NF and explain BCNF decomposition.	10
M4-Q07	List and explain informal design guidelines for relation schemas.	10
M4-Q08	Define functional dependency, full/partial/transitive dependency and Armstrong inference rules.	10
M4-Q09	Describe insertion, deletion and update anomalies with examples.	5
M4-Q10	Explain normal forms from 1NF through BCNF with suitable examples.	5
M4-Q11	Explain GROUP BY and HAVING with suitable SQL examples.	5
M4-Q12	Write five COMPANY SQL queries involving departments, projects, dependents, John Smith’s co-workers and project employee counts.	10
M4-Q13	Differentiate the WHERE, ORDER BY, DISTINCT, LIKE and BETWEEN clauses with examples.	5
M4-Q14	Explain SQL schema modification statements with examples.	5
M4-Q15	Demonstrate SQL constraints using NOT NULL, UNIQUE, PRIMARY KEY, FOREIGN KEY, DEFAULT and CHECK.	10
M4-Q16	Explain lossless-join and dependency-preserving decomposition.	10
M4-Q17	Demonstrate 3NF synthesis and BCNF decomposition procedures with an FD example.	10
M4-Q18	Explain why decomposition can generate spurious tuples and how to prevent them.	5
M4-Q19	Differentiate 1NF, 2NF, 3NF, BCNF, 4NF and 5NF at a glance.	5
M4-Q20	Explain aggregate functions and the effect of NULL in SQL.	5

Model answers and suggested marking elements

M4-Q01 | 10 marks

Question. Explain SQL data-definition commands and demonstrate CREATE TABLE with integrity constraints.

Suggested marking allocation: DDL/schema/types 3; CREATE TABLE 4; constraints 3.

DDL defines schema objects: CREATE SCHEMA, CREATE TABLE, ALTER TABLE, DROP TABLE. A schema groups related tables/views/constraints; a catalog is a collection of schemas. Typical SQL types include INT, CHAR(n), VARCHAR(n), DECIMAL(p,s), DATE and TIMESTAMP.

```
CREATE TABLE Department (
 Dnumber INT PRIMARY KEY,
 Dname VARCHAR(40) NOT NULL UNIQUE
);
CREATE TABLE Employee (
 Ssn CHAR(9) PRIMARY KEY,
 EmpName VARCHAR(40) NOT NULL,
 Salary DECIMAL(10,2) DEFAULT 0 CHECK (Salary >= 0),
 Dno INT NOT NULL,
 CONSTRAINT FK_EmpDept FOREIGN KEY (Dno)
  REFERENCES Department(Dnumber)
  ON DELETE RESTRICT
);
```

Constraints: PRIMARY KEY enforces unique/non-NULL identity; UNIQUE gives alternate uniqueness; NOT NULL disallows missing values; CHECK specifies row/domain rule; DEFAULT supplies omitted values; FOREIGN KEY requires an existing parent key. Creating parent table first avoids unresolved FK references. Constraints can also be declared with ALTER TABLE. State meaning of each keyword in an exam.

M4-Q02 | 5 marks

Question. Classify SQL data types with examples.

Suggested marking allocation: Five types with examples 5.

1. **Exact numeric:** INT, SMALLINT, DECIMAL(10,2), e.g., monetary salary.
2. **Approximate numeric:** REAL, FLOAT, DOUBLE PRECISION for measurements.
3. **Character strings:** CHAR(10) fixed length and VARCHAR(40) variable length.
4. **Date/time:** DATE, TIME, TIMESTAMP for birth date and transaction times.
5. **Boolean/binary and large objects:** BOOLEAN, BINARY/VARBINARY, BLOB/CLOB (availability and syntax vary by DBMS).

Employee(Ssn CHAR(9), Name VARCHAR(50), Salary DECIMAL(9,2), Bdate DATE) illustrates correct matching of domains to attributes. Mention SQL dialect differences when giving BLOB/BOOLEAN examples.

M4-Q03 | 10 marks

Question. Explain INSERT, DELETE and UPDATE SQL statements with suitable examples.

Suggested marking allocation: Three commands 6; variations 2; constraints/cautions 2.

INSERT adds rows. INSERT INTO Employee(Ssn,EmpName,Salary,Dno) VALUES ('100000001','Asha',50000,10); More than one row may be inserted using multiple VALUES tuples or INSERT ... SELECT ....

DELETE removes matching rows: DELETE FROM Employee WHERE Ssn='100000001';. Without WHERE, all rows are deleted while the table definition remains. Referential actions (RESTRICT/CASCADE/SET NULL) may affect dependent rows.

UPDATE changes existing values: UPDATE Employee SET Salary=Salary*1.10 WHERE Dno=10; updates every employee in department 10; without WHERE, every row is affected. Multiple columns can be assigned in one statement.

Validation: all DML statements must satisfy NOT NULL, key, CHECK and FK constraints. Deleting a referenced parent may be rejected; changing a PK referenced by child rows may cascade only if declared. In an exam, write general syntax as well: INSERT INTO T(cols) VALUES (...);, DELETE FROM T WHERE condition;, UPDATE T SET column=expr WHERE condition;. Explain that transaction rollback reverses uncommitted changes after failure.

M4-Q04 | 10 marks

Question. What is normalization? Explain 1NF, 2NF and 3NF with a complete example.

Suggested marking allocation: Definition/problems 1; 1NF 2; 2NF 3; 3NF 3; result 1.

Normalization organizes relations to reduce redundancy, update anomalies and undesirable dependencies, using functional dependencies (FDs) and lossless decomposition.

Suppose ENROLL(USN, StudentName, CourseID, CourseName, Instructor, Grade) has candidate key **(USN,CourseID)** with dependencies USN -> StudentName, CourseID -> CourseName,Instructor and (USN,CourseID) -> Grade.

1NF: all attribute values must be atomic; no repeating groups. Store one course per row rather than a comma-separated CourseList. The six-column ENROLL table with atomic cells is 1NF, though redundant.

2NF: relation is in 1NF and every nonprime attribute fully depends on each candidate key; remove partial dependencies on USN and CourseID: STUDENT(USN,StudentName), COURSE(CourseID,CourseName,Instructor), ENROLLMENT(USN,CourseID,Grade).

3NF: relation is in 2NF and has no transitive nonkey dependency (formal test: for every nontrivial FD X->A, X is a superkey or A is prime). If CourseID -> InstructorID and InstructorID -> InstructorName, decompose COURSE(CourseID,CourseName,InstructorID) and INSTRUCTOR(InstructorID,InstructorName).

Final schema: STUDENT, COURSE, INSTRUCTOR, ENROLLMENT; mark PKs, FKs and show these relations can be joined to reconstruct facts without accidental combinations.

M4-Q05 | 10 marks

Question. Explain multivalued dependencies, 4NF, join dependencies and 5NF with examples.

Suggested marking allocation: MVD/4NF 5; JD/5NF 4; lossless remark 1.

A **multivalued dependency (MVD)** $X \twoheadrightarrow Y$ in R(X,Y,Z) means that for one X, valid Y-values are independent of valid Z-values. Example EMP_INFO(Emp,Skill,Hobby): an employee's skills and hobbies vary independently; each skill-hobby combination repeats.

4NF: a relation is in 4NF if, for each nontrivial MVD $X \twoheadrightarrow Y$, X is a superkey. Since $Emp \twoheadrightarrow Skill$ and $Emp \twoheadrightarrow Hobby$ are nontrivial while Emp is not a superkey, decompose losslessly into EMP_SKILL(Emp,Skill) and EMP_HOBBY(Emp,Hobby). This eliminates cross-product redundancy.

A **join dependency (JD)** $*(R1,R2,...,Rn)$ states that relation R can be reconstructed exactly as the natural join of its projections onto R1,...,Rn. **5NF/PJNF:** every nontrivial JD is implied by candidate keys; the relation cannot be further losslessly decomposed because of a non-key JD. Classic **SUPPLY(Supplier,Part,Project)** may be decomposed into SP(Supplier,Part), SJ(Supplier,Project) and PJ(Part,Project) if the business rule makes their join exact. A JD needs careful real-world justification: blindly decomposing a ternary relationship may create spurious tuples.

M4-Q06 | 10 marks

Question. Define BCNF, compare it with 3NF and explain BCNF decomposition.

Suggested marking allocation: Definition 3; counterexample 3; decomposition 3; caveat 1.

BCNF definition: a relation R is in Boyce-Codd Normal Form if for every nontrivial FD $X \rightarrow Y$, **X is a superkey**. **3NF** permits $X \rightarrow A$ where X is not a superkey if A is a prime attribute (part of some candidate key); hence every BCNF relation is 3NF, not vice versa.

Example: TEACH(Student,Course,Instructor) with FDs $(Student,Course) \rightarrow Instructor$ and $Instructor \rightarrow Course$, assuming each instructor teaches one course. Candidate keys are (Student,Course) and (Student,Instructor), so Course is prime. This may satisfy 3NF but violates BCNF because Instructor is not a superkey.

Decompose: $IC(Instructor,Course)$ and $SI(Student,Instructor)$. The common attribute Instructor functionally determines IC, so the decomposition is **lossless**. However, the dependency $(Student,Course) \rightarrow Instructor$ may not be enforceable from the two relations separately; BCNF decomposition can sacrifice dependency preservation. **Conclusion:** 3NF synthesis often preserves dependencies; BCNF more aggressively removes redundancy.

M4-Q07 | 10 marks

Question. List and explain informal design guidelines for relation schemas.

Suggested marking allocation: Four guidelines 8; examples/conclusion 2.

1. **Clear semantics of attributes (2):** each relation should represent one entity/relationship theme. Do not mix unrelated employee, project and department facts without need; it obscures dependencies.
2. **Reduce redundancy and update anomalies (2):** store a fact such as Dname in DEPARTMENT rather than repeating it for every employee. Repetition produces insertion, deletion and modification anomalies.
3. **Avoid excessive NULLs (2):** put rarely applicable attributes in separate optional relations; e.g., DRIVER_LICENSE(EmpID,LicenseNo) instead of many NULL LicenseNo values in EMPLOYEE.
4. **Avoid spurious tuples; guarantee nonadditive/lossless joins (2):** a decomposition must reconstruct exactly the original relation; arbitrary projections may generate false combinations when rejoined. Example decompose EMP_PROJ by a nonkey shared attribute incorrectly.

Design test (2): identify meaningful functional dependencies and keys, choose 3NF/BCNF where suitable, verify lossless joins and dependency preservation, and test with sample inserts/updates/deletes. A good relational schema minimizes redundancy without losing real-world information.

M4-Q08 | 10 marks

Question. Define functional dependency, full/partial/transitive dependency and Armstrong inference rules.

Suggested marking allocation: FD and types 4; rules 3; closure/example 3.

FD: X -> Y means any two tuples agreeing on X must agree on Y. **Full FD:** Y depends on all of X and not a proper subset; e.g., (USN,CourseID) -> Grade. **Partial FD:** Y depends on a proper subset of a composite key; e.g., (USN,CourseID) -> StudentName because USN -> StudentName. **Transitive FD:** X -> Z and Z -> Y yield X -> Y with a nonkey intermediary (e.g., EmpID -> DeptID -> DeptName).

Armstrong's sound and complete axioms: (1) Reflexivity: Y⊆X implies X->Y; (2) Augmentation: X->Y implies XZ->YZ; (3) Transitivity: X->Y and Y->Z imply X->Z. Derived rules: union, decomposition and pseudotransitivity.

Closure example: R(A,B,C,D), F={A->B, B->C, AC->D}. Start A+={A}; apply A->B -> {A,B}; B->C -> {A,B,C}; AC->D -> {A,B,C,D}. Therefore A is a superkey. Test minimality before declaring a candidate key. Underline determinants and illustrate each dependency with a business fact.

M4-Q09 | 5 marks

Question. Describe insertion, deletion and update anomalies with examples.

Suggested marking allocation: Three anomalies 3; normalization solution 2.

Consider EMP_DEPT(EmpID,EmpName,DeptNo,DeptName,DeptPhone) where DeptNo -> DeptName,DeptPhone.

Insertion anomaly: cannot add a new department without inventing an employee if the table requires EmpID. **Deletion anomaly:** deleting the last employee in a department also removes the only recorded department phone. **Update anomaly:** changing a department phone requires changing many employee rows; missing one causes inconsistent facts. **Solution:** decompose into EMPLOYEE(EmpID,EmpName,DeptNo) and DEPARTMENT(DeptNo,DeptName,DeptPhone); add FK EMPLOYEE.DeptNo -> DEPARTMENT.DeptNo. Explain each with the same concrete table.

M4-Q10 | 5 marks

Question. Explain normal forms from 1NF through BCNF with suitable examples.

Suggested marking allocation: Four definitions/examples (1 mark each) + sequence (1).

1NF: every cell is atomic, no repeating groups; move StudentPhones(USN,Phone) to a separate table. **2NF:** 1NF and no partial dependency of nonprime attributes on a candidate key; split ENROLL(USN,Course,StudentName,Grade) into STUDENT(USN,StudentName), ENROLL(USN,Course,Grade). **3NF:** for every nontrivial FD X->A, X superkey or A prime; separate EmpID->DeptID->DeptName as EMPLOYEE(EmpID,DeptID) and DEPARTMENT(DeptID,DeptName). **BCNF:** every determinant in a nontrivial FD must be a superkey; TEACH(Student,Course,Instructor) with Instructor->Course violates BCNF if Instructor is not a superkey. Sequence: BCNF ⊂ 3NF ⊂ 2NF ⊂ 1NF (in terms of conditions).

M4-Q11 | 5 marks

Question. Explain GROUP BY and HAVING with suitable SQL examples.

Suggested marking allocation: GROUP BY 2; HAVING 2; contrast 1.

GROUP BY collects rows sharing group-by values; aggregate functions (COUNT, SUM, AVG, MIN, MAX) summarize each group. HAVING filters grouped results using an aggregate condition; WHERE filters individual rows **before** grouping.

```
SELECT Dno, COUNT(*) AS EmpCount, AVG(Salary) AS AvgSalary
FROM Employee
WHERE Salary > 0
GROUP BY Dno
HAVING COUNT(*) >= 3;
```

The query returns departments with at least three employees having positive salary, including their count and average. Logical processing order: FROM/JOIN → WHERE → GROUP BY → HAVING → SELECT → ORDER BY. Aggregate conditions such as COUNT(*) > 2 belong in HAVING, not WHERE.

M4-Q12 | 10 marks

Question. Using the given COMPANY database relations, write SQL queries for employees in department 5, projects of department 4, employees with no dependents, co-workers on John Smith's projects and employee counts by project.

Suggested marking allocation: Five executable queries, 2 each.

Assume Employee(Fname,Minit,Lname,Ssn,Bdate,Address,Sex,Salary,Super_ssn,Dno), Department(Dname,Dnumber,Mgr_ssn,Mgr_start_date), Project(Pname,Pnumber,Plocation,Dnum), Works_on(Essn,Pno,Hours), Dependent(Essn,Dependent_name,Sex,Bdate,Relationship).

(i) Names in department 5:

```
SELECT Fname, Lname FROM Employee WHERE Dno = 5;
```

(ii) Projects controlled by department 4:

```
SELECT Pnumber, Pname FROM Project WHERE Dnum = 4;
```

(iii) Employees who have no dependents:

```
SELECT e.Fname, e.Lname FROM Employee e
WHERE NOT EXISTS (SELECT 1 FROM Dependent d WHERE d.Essn=e.Ssn);
```

(iv) Employees working on every project on which John Smith works:

```
SELECT e.Fname, e.Lname FROM Employee e
WHERE NOT EXISTS (
 SELECT 1 FROM Works_on j
 JOIN Employee s ON s.Ssn=j.Essn
 WHERE s.Fname='John' AND s.Lname='Smith'
  AND NOT EXISTS (SELECT 1 FROM Works_on w
          WHERE w.Essn=e.Ssn AND w.Pno=j.Pno));
```

(v) Project number with employee count, including zero assignments:

```
SELECT p.Pnumber, COUNT(w.Essn) AS EmpCount
FROM Project p LEFT JOIN Works_on w ON w.Pno=p.Pnumber
GROUP BY p.Pnumber;
```

Explain **double NOT EXISTS** as “no Smith project is missing for the candidate employee.” For (iv), the literal question permits John Smith himself in the result and does not explicitly require at least one project.

M4-Q13 | 5 marks

Question. Differentiate the WHERE, ORDER BY, DISTINCT, LIKE and BETWEEN clauses with examples.

Suggested marking allocation: Five clauses 5.

WHERE filters rows: WHERE Salary>50000. **ORDER BY** sorts results: ORDER BY Salary DESC, EmpName ASC. **DISTINCT** eliminates duplicate result rows: SELECT DISTINCT Dno FROM Employee. **LIKE** performs pattern matching: WHERE EmpName LIKE 'A%' (starts with A), _ matches one character. **BETWEEN** tests an inclusive range: WHERE Salary BETWEEN 30000 AND 60000. Use IS NULL rather than = NULL for missing values. Combining clauses: SELECT DISTINCT Dno FROM Employee WHERE Salary>0 ORDER BY Dno;.

M4-Q14 | 5 marks

Question. Explain SQL schema modification statements with examples.

Suggested marking allocation: ALTER 2; DROP 2; cautions 1.

ALTER TABLE modifies existing definitions: ALTER TABLE Employee ADD Email VARCHAR(80);, ALTER TABLE Employee ADD CONSTRAINT chk_sal CHECK (Salary>=0);, and, where supported, DROP COLUMN Email / ALTER COLUMN type. DROP TABLE Employee; deletes the table definition and data; DROP SCHEMA s CASCADE; may remove dependent objects where supported. RESTRICT refuses removal when dependent objects exist, whereas CASCADE removes dependent objects (DBMS-dependent semantics). **Contrast:** DELETE FROM Employee removes rows, not the table schema. SQL dialects vary in ALTER syntax.

M4-Q15 | 10 marks

Question. Demonstrate SQL constraints using NOT NULL, UNIQUE, PRIMARY KEY, FOREIGN KEY, DEFAULT and CHECK.

Suggested marking allocation: Six correct constraints 6; table example 3; explanation 1.

```
CREATE TABLE Student (
  USN CHAR(10) PRIMARY KEY,
  Name VARCHAR(60) NOT NULL,
  Email VARCHAR(100) UNIQUE,
  Branch VARCHAR(10) DEFAULT 'CSE',
  Age INT CHECK (Age BETWEEN 16 AND 80)
);
CREATE TABLE Enrolment (
  USN CHAR(10) NOT NULL,
```

```
  CourseID CHAR(8) NOT NULL,
  Grade CHAR(2),
  PRIMARY KEY (USN, CourseID),
  FOREIGN KEY (USN) REFERENCES Student(USN)
    ON DELETE CASCADE
);
```

NOT NULL blocks missing Name; **UNIQUE** prevents duplicate non-NULL Email values (NULL semantics vary among systems); **PK** combines uniqueness and NOT NULL; **FK** enforces parent existence; **DEFAULT** fills omitted Branch; **CHECK** enforces age domain; **ON DELETE CASCADE** deletes a student's enrollments when the student is deleted. CourseID in Enrolment should additionally reference a COURSE table when that table exists. Mention row-level and column-level constraints.

M4-Q16 | 10 marks

Question. Explain lossless-join and dependency-preserving decomposition.

Suggested marking allocation: Definitions 4; lossless condition 3; examples/tradeoff 3.

A decomposition R -> R1,R2 is **lossless/nonadditive** if R = π_R1(R) ⋈ π_R2(R) for every valid instance satisfying dependencies; it creates **no spurious tuples**. For a binary decomposition, a sufficient and necessary FD test under F is (R1∩R2) -> R1 OR (R1∩R2) -> R2 in F+.

Example: EMP(EmpID,DeptID,DeptName) with DeptID -> DeptName decomposes to EMPLOYEE(EmpID,DeptID) and DEPARTMENT(DeptID,DeptName); common DeptID determines DEPARTMENT, so this is lossless.

Dependency preserving: all original FDs can be enforced by checking decomposed relations separately, without computing a join. If F+ = (F1 ∪ F2 ∪ ...)+ on the original attributes, it is dependency-preserving. **Importance:** losslessness protects information; preservation lowers constraint-checking costs. 3NF synthesis can achieve both, but BCNF may not preserve every FD. These are distinct design goals and must not be confused.

M4-Q17 | 10 marks

Question. Demonstrate 3NF synthesis and BCNF decomposition procedures with an FD example.

Suggested marking allocation: 3NF algorithm 4; BCNF 3; correctness/tradeoff 3.

3NF synthesis: (1) compute a minimal cover for F; (2) make a relation X ∪ Y for each FD X->Y; (3) remove any schema contained in another; (4) ensure at least one relation contains a candidate key of R (add it if not). The result is dependency-preserving and can be lossless.

Example: R(A,B,C), F={A->B,B->C}; minimal cover unchanged. Create R1(A,B) and R2(B,C); A is a candidate key and is in R1. Since shared B determines B,C, join is lossless.

BCNF decomposition: choose a violating nontrivial FD X->Y with X not a superkey; replace R with R1=X∪Y and R2=R−(Y−X). Repeat until each relation is BCNF. For R(A,B,C), same F, B->C violates BCNF; split into BC and AB. Both are BCNF. **Remember:** check losslessness; BCNF decomposition is lossless by construction but may lose FD preservation in other examples.

M4-Q18 | 5 marks

Question. Explain why decomposition can generate spurious tuples and how to prevent them.

Suggested marking allocation: Example 3; safeguard 2.

Let R(Student,Course,Instructor) contain (A,DBMS,X) and (B,OS,X). If we project to R1(Student,Instructor) and R2(Course,Instructor) and rejoin on Instructor, extra rows (A,OS,X) and (B,DBMS,X) appear. These are **spurious tuples**: combinations not in original R. Cause: shared Instructor does not functionally determine either original component for the given business facts. **Prevention:** ensure a lossless decomposition; for binary decomposition, intersection attributes must determine all attributes of at least one component (under the FDs). Do not decompose only to reduce column count.

M4-Q19 | 5 marks

Question. Differentiate 1NF, 2NF, 3NF, BCNF, 4NF and 5NF at a glance.

Suggested marking allocation: Six criteria summarized 4; hierarchy 1.

Normal form	Essential test
1NF	Atomic values and no repeating groups
2NF	1NF + no partial key dependency of a nonprime attribute
3NF	Every nontrivial X->A has X superkey or A prime
BCNF	Every nontrivial FD has a superkey determinant
4NF	Every nontrivial MVD has a superkey determinant
5NF	Every nontrivial join dependency is implied by candidate keys

Each higher normal form addresses an additional source of redundancy. **Caution:** full and partial dependencies must be checked against *all* candidate keys, not just the designated primary key.

M4-Q20 | 5 marks

Question. Explain aggregate functions and the effect of NULL in SQL.

Suggested marking allocation: Aggregates 3; NULL behavior 2.

COUNT(*) counts all rows; COUNT(column) counts non-NULL values; COUNT(DISTINCT column) counts distinct non-NULL values. SUM, AVG, MIN, MAX ignore NULL input values, although an aggregate may return NULL when no non-NULL value exists (except COUNT, which returns zero). Example SELECT Dno, COUNT(*) TotalRows, COUNT(Salary) KnownSalaries, AVG(Salary) AvgKnownSalary FROM Employee GROUP BY Dno;. Average salary is based on known salaries, not on treating NULL as zero.

MODULE 5: Advanced SQL and Concurrency Control

22 fully answered questions in this module.

Question index

ID	Question	Marks
M5-Q01	Explain NULL and SQL three-valued logic with truth tables and examples.	5
M5-Q02	Explain nested SQL queries and IN, ANY/SOME and ALL operators.	10
M5-Q03	Explain correlated subqueries, EXISTS and NOT EXISTS, including a universal-quantification query.	10
M5-Q04	Explain different joins in SQL with one example of each.	5
M5-Q05	Illustrate views in SQL with syntax, example, uses and limitations.	10
M5-Q06	Explain SQL assertions and triggers with suitable examples.	10
M5-Q07	Summarize the need for concurrency control in DBMS.	5
M5-Q08	Discuss Two-Phase Locking (2PL) techniques for concurrency control, including an example.	10
M5-Q09	Explain timestamp-ordering concurrency control with read and write rules.	10
M5-Q10	Explain Multiversion Concurrency Control (MVCC) and its advantages/disadvantages.	10
M5-Q11	Explain validation-based (optimistic) concurrency control in detail.	10
M5-Q12	Explain granularity of data items and multiple-granularity locking.	10
M5-Q13	Explain transaction ACID properties, serial schedules and conflict serializability.	5
M5-Q14	Explain deadlock and starvation in lock-based concurrency control.	5
M5-Q15	Check a two-transaction schedule for conflict serializability using a precedence graph.	10
M5-Q16	Write five SQL queries using Employee, Department and Project relations, including joins and departmental counts.	5
M5-Q17	Write five SQL queries using Student, Class, Enrolled and Faculty, including grouping, filtering and NOT EXISTS.	5
M5-Q18	Explain GROUP BY, HAVING, COUNT, SUM and AVG in advanced SQL.	5
M5-Q19	Explain WITH (CTE), CASE and recursive SQL queries with examples.	10
M5-Q20	Explain multiversion timestamp ordering and multiversion two-phase locking.	5
M5-Q21	Differentiate dirty read, lost update, nonrepeatable read, phantom read and write skew.	5
M5-Q22	Differentiate pessimistic, optimistic, timestamp and multiversion concurrency	5

ID	Question	Marks
	approaches.	

Model answers and suggested marking elements

M5-Q01 | 5 marks

Question. Explain NULL and SQL three-valued logic with truth tables and examples.

Suggested marking allocation: Meaning 1; truth behavior 2; SQL example 2.

NULL represents unknown, unavailable or inapplicable data. Comparisons such as Salary = NULL evaluate to **UNKNOWN**, not TRUE; use IS NULL / IS NOT NULL. SQL logical results are **TRUE, FALSE, UNKNOWN**. TRUE AND UNKNOWN=UNKNOWN; FALSE AND UNKNOWN=FALSE; TRUE OR UNKNOWN=TRUE; FALSE OR UNKNOWN=UNKNOWN; NOT UNKNOWN=UNKNOWN. **WHERE retains only TRUE rows**, excluding FALSE and UNKNOWN. Example:

```
SELECT EmpName FROM Employee WHERE Super_ssn IS NULL;
```

This correctly finds employees without supervisors. COUNT(Salary) ignores NULL while COUNT(*) counts rows. For an exam diagram, draw a three-row NOT table or a 3×3 AND table.

M5-Q02 | 10 marks

Question. Explain nested SQL queries and IN, ANY/SOME and ALL operators.

Suggested marking allocation: Nested concept 2; operators 4; examples 3; NULL caveat 1.

A **nested query/subquery** is a SELECT block inside another SQL query. It may return a scalar, a list of values or a relation. IN tests membership; = ANY/= SOME is equivalent to IN (subject to SQL NULL logic). > ANY means greater than at least one returned non-NULL value; > ALL means greater than every returned value.

```
-- Employees in departments that handle a project named Alpha
SELECT e.EmpName FROM Employee e
WHERE e.Dno IN (SELECT p.Dnum FROM Project p
        WHERE p.Pname='Alpha');

-- Employees earning more than every salary in department 20
SELECT e.EmpName FROM Employee e
WHERE e.Salary > ALL (SELECT x.Salary FROM Employee x
            WHERE x.Dno=20 AND x.Salary IS NOT NULL);
```

Notes: IN is suitable for a set-valued result, while = needs a single scalar. With an empty subquery, > ALL is vacuously TRUE; > ANY is FALSE. NULLs can make comparisons UNKNOWN. Prefer NOT EXISTS for anti-joins when nullable subquery columns are involved; NOT IN can behave unexpectedly with NULL.

M5-Q03 | 10 marks

Question. Explain correlated subqueries, EXISTS and NOT EXISTS, including a universal-quantification query.

Suggested marking allocation: Correlation 2; EXISTS 2; NOT EXISTS 2; universal query 4.

A **correlated subquery** references an attribute of the surrounding query and is logically evaluated for candidate outer rows. EXISTS(subquery) is TRUE if the subquery returns at least one row, while NOT EXISTS is TRUE if it returns none.

```
-- Employees with at least one dependent
SELECT e.Fname, e.Lname FROM Employee e
WHERE EXISTS (SELECT 1 FROM Dependent d WHERE d.Essn=e.Ssn);

-- Employees with no dependent
SELECT e.Fname, e.Lname FROM Employee e
WHERE NOT EXISTS (SELECT 1 FROM Dependent d WHERE d.Essn=e.Ssn);
```

For-all pattern: "employee works on every project of department 5" means no required project exists for which that employee lacks a corresponding WORKS_ON row:

```
SELECT e.Ssn FROM Employee e WHERE NOT EXISTS (
 SELECT 1 FROM Project p WHERE p.Dnum=5 AND NOT EXISTS (
  SELECT 1 FROM Works_on w
  WHERE w.Essn=e.Ssn AND w.Pno=p.Pnumber));
```

Explain nested double negation. If no department-5 projects exist, the condition is true for every employee under standard logic.

M5-Q04 | 5 marks

Question. Explain different joins in SQL with one example of each.

Suggested marking allocation: Inner 2; outer 2; cross/self 1.

INNER JOIN returns matching rows: SELECT * FROM Employee e JOIN Department d ON e.Dno=d.Dnumber;. **LEFT JOIN** preserves all employees including those without a matching department; right columns become NULL. **RIGHT JOIN** preserves all right-table rows; **FULL OUTER JOIN** preserves matched and unmatched rows from both sides (not supported by every SQL engine). **CROSS JOIN** returns Cartesian pairs: FROM Employee CROSS JOIN Department. **SELF JOIN** compares one table with itself using aliases: Employee e JOIN Employee boss ON e.Super_ssn=boss.Ssn. Use ON conditions to avoid accidental Cartesian products.

M5-Q05 | 10 marks

Question. Illustrate views in SQL with syntax, example, uses and limitations.

Suggested marking allocation: Definition/syntax 2; worked view 3; uses 3; limitations 2.

A **view** is a named virtual table defined by a stored SELECT query; its result is normally computed when referenced rather than stored as independent base rows.

```
CREATE VIEW DeptSalary AS
SELECT d.Dnumber, d.Dname,
     COUNT(e.Ssn) AS EmpCount, AVG(e.Salary) AS AvgSalary
FROM Department d LEFT JOIN Employee e
 ON e.Dno=d.Dnumber
GROUP BY d.Dnumber,d.Dname;
```

```
SELECT Dname, AvgSalary FROM DeptSalary WHERE AvgSalary > 50000;
-- Drop definition, not base rows:
DROP VIEW DeptSalary;
```

Advantages: (1) simplifies repeated joins/queries; (2) presents selected columns/rows for security, together with proper privileges; (3) provides logical data independence and a stable interface; (4) supports customized reports. **Limits:** joins/aggregates/DISTINCT often make views non-updatable; performance depends on query planning; a changed base schema can invalidate views. **WITH CHECK OPTION** on a suitable updatable view requires updates to remain visible in that view. **Materialized views** store a result physically and require refresh; unlike normal views. For a 5-mark answer, write definition, CREATE VIEW example, two advantages and one limitation.

M5-Q06 | 10 marks

Question. Explain SQL assertions and triggers with suitable examples.

Suggested marking allocation: Assertion 3; trigger 3; examples 3; comparison/limitations 1.

An **assertion** is a schema-level integrity condition that must remain TRUE for all valid database states, possibly spanning several tables. The SQL-standard idea is:

```
CREATE ASSERTION SalaryNonnegative
CHECK (NOT EXISTS (SELECT 1 FROM Employee WHERE Salary < 0));
```

Important: CREATE ASSERTION is in SQL standards but not implemented by many common DBMS products; it is a conceptual syntax example. Use CHECK for row-local rules where possible.

A **trigger** is an event-condition-action rule automatically executed on INSERT, UPDATE or DELETE. Typical components: timing (BEFORE/AFTER), event, table, optional row/statement granularity, optional WHEN predicate, and action.

```
-- Illustrative standard-style pseudocode; exact syntax varies by DBMS
CREATE TRIGGER AuditSalary
AFTER UPDATE OF Salary ON Employee
REFERENCING OLD ROW AS oldrow NEW ROW AS newrow
FOR EACH ROW
WHEN (oldrow.Salary <> newrow.Salary)
 INSERT INTO SalaryAudit(Ssn,OldSalary,NewSalary)
 VALUES (newrow.Ssn,oldrow.Salary,newrow.Salary);
```

Difference: an assertion expresses an invariant; a trigger performs an action when an event occurs (validation, audit or propagation). Trigger syntax differs among PostgreSQL/MySQL/Oracle; write the required dialect for runnable production SQL.

M5-Q07 | 5 marks

Question. Summarize the need for concurrency control in DBMS.

Suggested marking allocation: Purpose 1; anomalies 3; mechanisms 1.

Multiple transactions execute simultaneously for better throughput and resource utilization, but their interleaved actions can cause **lost updates** (one write overwrites another), **dirty reads** (read of data not yet committed), **nonrepeatable reads** (same row changes between reads) and **phantoms** (re-running a predicate query returns a different matching set). Concurrency control maintains isolation and consistency while preventing unsafe interference. Methods include **two-phase locking**, **timestamp ordering**, **multiversion**

concurrency control and **optimistic validation**. **Example:** if T1 and T2 both read balance 1000 and separately write 1100 and 900, one update can be lost; appropriate isolation/locking prevents the conflicting schedule.

M5-Q08 | 10 marks

Question. Discuss Two-Phase Locking (2PL) techniques for concurrency control, including an example.

Suggested marking allocation: Definition 2; phases/diagram 3; variants 3; pros/limitations 2.

A lock restricts access to a data item. **S/shared lock** permits multiple readers; **X/exclusive lock** is needed for writing and conflicts with other locks on that item. **2PL rule:** every transaction has two phases: **growing** (acquire locks, release none) and **shrinking** (release locks, acquire none). Once a lock is released, no new lock may be acquired by that transaction.

```
Transaction T1: LOCK-X(A) -> READ A -> WRITE A
        -> LOCK-S(B) -> READ B -> UNLOCK A
        -> UNLOCK B -> COMMIT
        ^ growing ^      ^ shrinking ^
```

Guarantee: basic 2PL produces conflict-serializable schedules, but can deadlock and does not by itself prevent cascading rollback. Strict 2PL: retains all X locks until commit/abort, avoids dirty writes and cascading aborts. Rigorous 2PL: retains both S and X locks until commit/abort. Conservative/static 2PL: obtains all needed locks before work, avoiding deadlock due to lock acquisition (but needs advance knowledge and may reduce concurrency). Illustration: T1 locks A in X mode; T2 requesting X(A) must wait until T1 releases it. For a 5-mark response, prioritize S/X meaning, two phases, a lock timeline, guarantee, and one drawback.

M5-Q09 | 10 marks

Question. Explain timestamp-ordering concurrency control with read and write rules.

Suggested marking allocation: Timestamp idea 2; data timestamps 2; read/write rules 4; pros/cons 2.

Each transaction T gets a unique timestamp TS(T). Each data item X tracks **read_TS(X)** (largest TS of a transaction that read X) and **write_TS(X)** (largest TS that wrote X). The protocol enforces the ordering imposed by timestamps.

READ(T,X): if TS(T) < write_TS(X), reject/abort T because a newer transaction already wrote X; otherwise allow read and set read_TS(X)=max(read_TS(X),TS(T)).

WRITE(T,X): if TS(T) < read_TS(X), reject/abort T; otherwise, if TS(T) < write_TS(X), basic timestamp ordering rejects T's obsolete write (Thomas write rule may ignore it under a different variant); else write X and set write_TS(X)=TS(T).

Example: TS(T1)=1 and TS(T2)=2. If T2 writes X first (write_TS(X)=2), a later T1 read of X is rejected because 1<2. **Benefits:** no lock waiting/deadlock; **limitations:** abort/restart and possible starvation, timestamp metadata overhead. Distinguish plain timestamp ordering from multiversion timestamp ordering.

M5-Q10 | 10 marks

Question. Explain Multiversion Concurrency Control (MVCC) and its advantages/disadvantages.

Suggested marking allocation: Definition 2; versions/read-write 3; example 2; benefits/limits 3.

MVCC maintains multiple committed or pending versions of data items instead of overwriting every old value. Transactions may read a version appropriate to their snapshot/time, allowing readers and writers to operate with less blocking.

Example: balance version v1=1000 was committed at t1. T2 begins with snapshot t1; T3 later updates balance to 1200 and commits at t2. Depending on the isolation implementation, T2 continues reading the older committed v1=1000; a new transaction sees v2=1200. **Version metadata** may include creator transaction ID, start/end timestamps and visibility information.

Benefits: high read concurrency, consistent snapshot reads, fewer read-write waits. **Costs:** additional storage, cleanup/vacuum of stale versions, version visibility complexity, and possible write conflicts. **Important limitation:** MVCC alone does not imply full serializability; snapshot isolation can permit anomalies such as write skew unless extra checks or serializable isolation are used. Contrast with 2PL, which often blocks competing operations using S/X locks.

M5-Q11 | 10 marks

Question. Explain validation-based (optimistic) concurrency control in detail.

Suggested marking allocation: Three phases 6; validation checks 2; strengths/limitations 2.

Optimistic concurrency control (OCC) assumes conflicts are relatively rare. Each transaction computes on private/local copies without retaining long-duration locks; at commit it verifies that its result is safe.

1. Read phase: read database items, perform local computations and collect read set RS(T), write set WS(T). **2. Validation phase:** before publishing writes, check whether concurrent transactions violate the chosen serial order. Typical tests compare read/write sets with transactions that committed or overlap during T's execution. **3. Write phase:** if validation succeeds, apply buffered updates atomically and commit; otherwise abort and restart.

```
T1:  READ/compute -------- VALIDATE -- WRITE -- COMMIT
T2:       READ/compute ------ VALIDATE (conflict?)
                                -> RESTART if invalid
```

Illustrative conflict: T1 reads X; T2 writes X and commits before T1 validates. If the intended serialization would make T1's read stale, validation fails and T1 restarts. **Benefits:** no lock deadlocks, strong performance when conflicts are rare. **Drawbacks:** repeated work under high contention, starvation/retries and validation overhead. Do not claim all OCC implementations use identical validation formulas.

M5-Q12 | 10 marks

Question. Explain granularity of data items and multiple-granularity locking.

Suggested marking allocation: Granularity 2; hierarchy 2; intention locks 3; protocol/example 3.

Lock granularity is the size of a locked item: database, table, page, record or field. **Fine-grained locks** permit more concurrency but cost more lock-table overhead; **coarse-grained locks** cost less but reduce concurrency.

```
DATABASE
  |
TABLE (EMPLOYEE)
  |
PAGE / BLOCK
  |
RECORD (employee row)
```

Multiple-granularity locking uses intention locks on ancestors to advertise descendant locking. **IS** means intention to take S on descendants; **IX** means intention to take X; **SIX** means shared lock on the node plus intention to X-lock descendants. Simplified compatibility:

	IS	IX	S	SIX	X
IS	Y	Y	Y	Y	N
IX	Y	Y	N	N	N
S	Y	N	Y	N	N
SIX	Y	N	N	N	N
X	N	N	N	N	N

Protocol: lock root/ancestors with appropriate intention locks before locking a child; unlock descendants before ancestors. **Example:** to X-lock one employee row, take IX(Database), IX(Employee table), then X(Row). A table-level S lock conflicts with another transaction's table IX, preventing unsafe mixed-granularity access.

M5-Q13 | 5 marks

Question. Explain transaction ACID properties, serial schedules and conflict serializability.

Suggested marking allocation: ACID 2; schedules 2; graph 1.

Atomicity, Consistency, Isolation, Durability are transaction properties. A **serial schedule** completes one transaction before starting the next; a **concurrent schedule** interleaves operations. A schedule is **conflict-serializable** if it can be transformed into a serial schedule by swapping nonconflicting operations. Build a **precedence graph** with one vertex per transaction; draw edge Ti->Tj when a conflicting operation of Ti occurs before an operation of Tj on the same item (at least one write). **Acyclic** precedence graph means conflict serializable; any directed cycle means not conflict serializable.

M5-Q14 | 5 marks

Question. Explain deadlock and starvation in lock-based concurrency control.

Suggested marking allocation: Deadlock 2; management 2; starvation 1.

Deadlock: T1 holds X(A) and waits for X(B), while T2 holds X(B) and waits for X(A); neither can progress. **Detection:** build a wait-for graph; a cycle indicates deadlock. **Prevention:** impose lock ordering, use wait-die/wound-wait based on timestamps, or acquire all locks in advance (conservative 2PL). **Recovery:** abort/rollback a chosen victim, releasing its locks. **Starvation:** a transaction is repeatedly delayed or selected as victim; mitigate with fair queues, priorities/aging and bounded retries. Note that basic 2PL can produce deadlocks, whereas timestamp ordering has aborts without lock deadlocks.

M5-Q15 | 10 marks

Question. Check a two-transaction schedule for conflict serializability using a precedence graph.

Suggested marking allocation: Conflict pairs 4; graph 3; verdict 2; contrast 1.

Given S = r1(X), w1(X), r2(X), w2(Y), r1(Y), w1(Y), construct precedence edges for conflicting operations on the same item.

Item X: w1(X) appears before r2(X) -> edge **T1 -> T2**. **Item Y:** w2(Y) appears before r1(Y) and w1(Y) -> edge **T2 -> T1**. Hence graph T1 -> T2 -> T1 contains a cycle. **Conclusion:** the schedule is **not conflict-serializable**; it is not equivalent to any serial ordering by swapping nonconflicting operations. (Two reads of the same item do not conflict.) **Alternative serializable schedule:** r1(X),w1(X),r1(Y),w1(Y),r2(X),w2(Y) gives only T1 -> T2. Draw two labeled circles and directed arrows for graph marks.

M5-Q16 | 5 marks

Question. Given Employee(SSN,EmpName,role,salary,city,Dnum), Department(DNo,Dname,Location) and Project(Pid,Pname,Dnum), write five SQL queries: (1) employee names with department and location, (2) Bangalore residents, (3) salary in department 10, (4) per-project employee counts in the associated department, (5) project counts by department.

Suggested marking allocation: Five correct SQL statements, 1 each.

Schemas: Employee(SSN,EmpName,role,salary,city,Dnum), Department(DNo,Dname,Location), Project(Pid,Pname,Dnum).

1. Employees with department names and locations:

```
SELECT e.EmpName,d.Dname,d.Location FROM Employee e
JOIN Department d ON e.Dnum=d.DNo;
```

2. Names of Bangalore residents:

```
SELECT EmpName FROM Employee WHERE city='Bangalore';
```

3. Salaries in department 10:

```
SELECT EmpName,salary FROM Employee WHERE Dnum=10;
```

4. Each project and employee count in its department:

```
SELECT p.Pid,p.Pname,COUNT(e.SSN) AS EmpCount
FROM Project p LEFT JOIN Employee e ON e.Dnum=p.Dnum
GROUP BY p.Pid,p.Pname;
```

5. Departments and total projects including zero:

```
SELECT d.DNo,d.Dname,COUNT(p.Pid) AS ProjectCount
FROM Department d LEFT JOIN Project p ON p.Dnum=d.DNo
GROUP BY d.DNo,d.Dname;
```

Important: Q7(4) counts employees of the project's **department**, not employees personally assigned to that project (no WORKS_ON table is provided).

M5-Q17 | 5 marks

Question. Given Student(Snum,Sname,Branch,level,age), Class(Cname,meet_at,room,fid), Enrolled(Snum,Cname) and Faculty(fid,fname,deptid,level), write SQL for: (A) juniors in I. Teach's class, (B) classes in R128 or with at least five students, (C) mean age by level excluding JR, (D) faculty teaching only in R128, (E) students not enrolled in a class.

Suggested marking allocation: Five correct SQL statements, 1 each.

Student(Snum,Sname,Branch,level,age); Class(Cname,meet_at,room,fid); Enrolled(Snum,Cname); Faculty(fid,fname,deptid,level).

A. Juniors enrolled in a class taught by I. Teach:

```
SELECT DISTINCT s.Sname FROM Student s
JOIN Enrolled e ON e.Snum=s.Snum
JOIN Class c ON c.Cname=e.Cname
JOIN Faculty f ON f.fid=c.fid
WHERE s.level='JR' AND f.fname='I. Teach';
```

B. Classes meeting in R128 or having at least five students:

```
SELECT c.Cname FROM Class c WHERE c.room='R128'
OR (SELECT COUNT(*) FROM Enrolled e WHERE e.Cname=c.Cname)>=5;
```

C. Average age per level excluding JR:

```
SELECT level,AVG(age) AS AvgAge FROM Student
WHERE level<>'JR' GROUP BY level;
```

D. Faculty who taught one or more classes and *only* in R128:

```
SELECT f.fname,COUNT(*) AS ClassCount FROM Faculty f
JOIN Class c ON c.fid=f.fid
GROUP BY f.fid,f.fname
HAVING SUM(CASE WHEN c.room<>'R128' OR c.room IS NULL
          THEN 1 ELSE 0 END)=0;
```

E. Students enrolled in no class:

```
SELECT s.Sname FROM Student s
WHERE NOT EXISTS (SELECT 1 FROM Enrolled e WHERE e.Snum=s.Snum);
```

M5-Q18 | 5 marks

Question. Explain GROUP BY, HAVING, COUNT, SUM and AVG in advanced SQL.

Suggested marking allocation: Syntax 2; example 2; HAVING vs WHERE 1.

GROUP BY partitions rows into groups; aggregate functions compute per-group results. COUNT(*) counts rows, SUM(Salary) totals pay and AVG(Salary) gives the mean of non-NULL salaries. **HAVING** filters groups after aggregation; **WHERE** filters individual rows before grouping.

```
SELECT Dno,COUNT(*) AS N, SUM(Salary) AS Payroll,
    AVG(Salary) AS AveragePay
FROM Employee
WHERE Salary IS NOT NULL
GROUP BY Dno
HAVING COUNT(*)>=2;
```

The result includes only departments with at least two employees having known salaries. A selected nonaggregated attribute should belong to GROUP BY unless the SQL dialect permits a functionally dependent exception.

M5-Q19 | 10 marks

Question. Explain WITH (CTE), CASE and recursive SQL queries with examples.

Suggested marking allocation: WITH 3; CASE 3; recursive query 4.

WITH / Common Table Expression (CTE): names a temporary query result for the following statement, improving readability.

```
WITH DeptAvg AS (
  SELECT Dno, AVG(Salary) AS AvgPay FROM Employee GROUP BY Dno
)
SELECT e.EmpName FROM Employee e
JOIN DeptAvg d ON e.Dno=d.Dno WHERE e.Salary>d.AvgPay;
```

CASE returns a conditional expression:

```
SELECT EmpName, CASE WHEN Salary>=80000 THEN 'High'
  WHEN Salary>=40000 THEN 'Medium' ELSE 'Entry' END AS PayBand
FROM Employee;
```

Recursive CTE finds hierarchical paths such as a manager chain (standard-style SQL):

```
WITH RECURSIVE Tree(Ssn,Super_ssn,Depth) AS (
  SELECT Ssn,Super_ssn,0 FROM Employee WHERE Ssn='100000001'
  UNION ALL
  SELECT e.Ssn,e.Super_ssn,t.Depth+1
  FROM Employee e JOIN Tree t ON e.Super_ssn=t.Ssn
)
SELECT * FROM Tree;
```

Recursive CTE includes an **anchor** query, **UNION ALL**, a **recursive** member and a termination condition (no more matching rows; cycles must be controlled as needed). SQL dialect syntax may differ.

M5-Q20 | 5 marks

Question. Explain multiversion timestamp ordering and multiversion two-phase locking.

Suggested marking allocation: MVTO 2; MV2PL 2; comparison 1.

Multiversion Timestamp Ordering (MVTO): versions are associated with timestamps; read chooses the newest version whose write timestamp does not exceed the transaction timestamp, subject to validity checks. Writes may be rejected if they would violate a required timestamp order. **Multiversion 2PL (MV2PL):** combines versioned reads with lock-based rules to control updates and maintain a chosen serial order; exact mechanisms depend on implementation. **Comparison:** MVTO orders primarily by timestamps and may restart transactions; MV2PL relies on locking for write coordination. Both permit more read concurrency than single-version exclusive locking in relevant workloads.

M5-Q21 | 5 marks

Question. Differentiate dirty read, lost update, nonrepeatable read, phantom read and write skew.

Suggested marking allocation: Five correct anomalies with examples 5.

Dirty read: T2 reads T1's uncommitted value, later rolled back. **Lost update:** T1 and T2 read the same balance and one overwrites the other's update. **Nonrepeatable read:** T1 reads a row twice and finds a different committed value because T2 updated it. **Phantom:** T1 repeats WHERE Salary>50000 and sees a different set of matching rows due to T2 insert/delete. **Write skew:** under some snapshot-isolation systems, two transactions

read the same invariant and update different rows, jointly violating a constraint; neither has a direct write-write conflict. Appropriate isolation and constraints must be chosen.

M5-Q22 | 5 marks

Question. Differentiate pessimistic, optimistic, timestamp and multiversion concurrency approaches.

Suggested marking allocation: Four methods and tradeoff 5.

Method	Main idea	Typical concern
Pessimistic / 2PL	Acquire S/X locks before accesses	Waiting, deadlocks
Optimistic / validation	Work speculatively, validate before commit	Restarts when conflicts are frequent
Timestamp ordering	Enforce global timestamp-based operation order	Aborts of stale transactions
MVCC	Keep multiple versions for snapshot reads	Storage/cleanup and isolation subtleties

Use 2PL under contention with explicit lock management, OCC where conflicts are rare, and MVCC for high read concurrency, while remembering that isolation guarantees depend on the exact implementation and configuration.

APPENDIX A. PRIORITY REVISION PLAN AND SELF-ASSESSMENT

Priority revision topics across Modules 3, 4 and 5:

1. Module 3: relational model constraints and update violations; division; joins; COMPANY relational algebra queries; grouped aggregates and applied expressions.
2. Module 4: SQL INSERT, DELETE, UPDATE and DDL; 1NF, 2NF, 3NF, BCNF and higher normal forms; anomalies; dependencies; COMPANY SQL; GROUP BY and HAVING.
3. Module 5: views, assertions, triggers, advanced SQL, concurrency anomalies, two-phase locking, timestamp ordering, multiversion and validation techniques.

Additional practice: relational keys, SQL constraints, query trees, functional-dependency closure, MVD/4NF/JD/5NF, timestamp ordering, MVCC, multiple-granularity locks, correlated subqueries, WITH/CASE and isolation anomalies.

Student self-check: A 5/5 response should answer every requested subpart, use correct technical terms, show a relevant example when asked, and avoid contradictory statements. For 10 marks, a neat labelled diagram/table/SQL block should be combined with clearly explained steps and a result/conclusion. Verify actual SQL execution in the stated DBMS dialect; standard-style trigger/assertion syntax is not universally portable.

APPENDIX B. TWO PRACTICE PAPERS — MODULES 3, 4 AND 5

Each practice paper contains six full questions: two 20-mark alternatives from each of Modules 3, 4 and 5. Answer one full question from each module, for a 60-mark focused practice exercise. These are revision papers, not complete 100-mark university examination papers.

PRACTICE PAPER A — 60 MARKS

Question	Module	Subquestions (20 marks total)
1	3	(a) Relational constraints [10] M3-Q02; (b) COMPANY algebra queries [10] M3-Q09
2	3	(a) Constraint violations in updates [10] M3-Q03; (b) division [10] M3-Q07
3	4	(a) SQL INSERT/UPDATE/DELETE [10] M4-Q03; (b) normalization 1NF–3NF [10] M4-Q04
4	4	(a) Relation design guidelines [10] M4-Q07; (b) COMPANY SQL queries [10] M4-Q12
5	5	(a) Triggers/assertions [10] M5-Q06; (b) 2PL [10] M5-Q08
6	5	(a) Views [10] M5-Q05; (b) optimistic validation [10] M5-Q11

PRACTICE PAPER B — 60 MARKS

Question	Module	Subquestions (20 marks total)
1	3	(a) Relational model concepts [10] M3-Q01; (b) joins [10] M3-Q06
2	3	(a) Query trees [10] M3-Q12; (b) division [10] M3-Q07
3	4	(a) Functional dependencies [10] M4-Q08; (b) BCNF [10] M4-Q06
4	4	(a) 4NF/5NF [10] M4-Q05; (b) SQL integrity definitions [10] M4-Q01
5	5	(a) Timestamp ordering [10] M5-Q09; (b) MVCC [10] M5-Q10
6	5	(a) Nested SQL [10] M5-Q02; (b) multiple-granularity locking [10] M5-Q12

APPENDIX C. TWO PRACTICE TESTS — 25 MARKS EACH

Each practice test contains five groups of 5 marks, with a choice between Option A and Option B in every group. Answer one option per group. All questions are from Modules 3, 4 and 5.

PRACTICE TEST A (25 marks; 1 hour)

Group	Option A, 5 marks	OR: Option B, 5 marks
1	Five relational algebra COMPANY expressions (M3-Q09: answer in 5 brief parts)	Explain SELECT/PROJECT/RENAME with examples (M3-Q04)
2	GROUP BY and HAVING (M4-Q11)	Insertion, update and deletion anomalies (M4-Q09)
3	Different normal forms up to BCNF (M4-Q10)	Discuss SQL data types (M4-Q02)
4	Five Student/Class SQL queries (M5-Q17)	Five Employee/Project SQL queries (M5-Q16)
5	Importance of concurrency control (M5-Q07)	Demonstrate 2PL (M5-Q08, condensed)

PRACTICE TEST B (25 marks; 1 hour)

Group	Option A, 5 marks	OR: Option B, 5 marks
1	Relational keys with examples (M3-Q13)	Union compatibility and basic operations (M3-Q17)
2	SQL schema modification (M4-Q14)	DISTINCT, WHERE, LIKE, BETWEEN, ORDER BY (M4-Q13)
3	Normal forms from 1NF through BCNF (M4-Q10)	NULL and aggregate functions (M4-Q20)
4	Explain a view with CREATE VIEW example (M5-Q05, condensed)	Nested queries with IN/ANY/ALL (M5-Q02, condensed)
5	Timestamp ordering (M5-Q09, condensed)	Deadlocks and remedies (M5-Q14)